L2A. Views and Database Abstraction

Database Management Systems

Key problem

How can a DBMS provide a simplified, restricted, reusable representation of underlying data?

L2A Learning Outcomes

- **Define a view correctly.**
  - A view is a named virtual relation defined by a query.
  - The regular view result is not stored as an independent relation.
- **Use views in SQL queries.**
  - Write CREATE VIEW statements.
  - Query views as if they were relations.
- **Explain why views support abstraction and authorization.**
  - Expose only selected rows/columns.
  - Hide parts of the underlying schema.
- **Reason about update semantics.**
  - Recognize ambiguous view updates.
  - Apply basic updatability conditions and WITH CHECK OPTION.
- **Inspect and test views in MySQL.**
  - SHOW CREATE VIEW, INFORMATION_SCHEMA.VIEWS, CHECK TABLE.
  - Safe update experiments with transaction rollback.

Connection With Lecture 1



- **In L1, a SQL statement reached the DBMS as a running system.**
  - The DBMS checks metadata, permissions, and query definitions before execution.
- **In this lecture, a view becomes a stored database object.**
  - The object is stored in the catalog.
  - The object defines a reusable logical interface over base data.
- **The same query processor still executes the final query.**
  - Views do not remove query processing; they add a layer of abstraction.

Motivating Problem

A clerk needs instructor ID, name, and department. The clerk must not see salary.

```
SELECT ID, name, dept_name
FROM instructor;
```

- **Without a view:**
  - The application repeats the same projection query.
  - Direct table access may reveal sensitive columns.
- **With a view:**
  - The DBMS exposes a named virtual relation.
  - Users can be granted access to the view instead of the base relation.
- **This is a database-management problem:**
  - It involves abstraction, authorization, and schema-level interface design.

Part I - View Fundamentals

Definition, syntax, examples, and evaluation semantics.

- A view is a schema object defined by a query.
- A regular view is virtual; its result is not stored as an independent table.
- A view can hide complexity and provide a stable interface.

Source: DSC 7e, Ch.4 §4.2

Definition

A view is a named virtual relation defined by a query expression.

- **The DBMS stores the view definition in the catalog.**
  - It stores the query associated with the view name.
  - It does not store the regular view result as a separate base relation.
- **When the view is referenced, the DBMS computes the required result.**
  - Conceptually, the view behaves like a relation in a query.
  - Physically, the DBMS may **rewrite**, **merge**, or otherwise **optimize** the query.
- **A view remains available until it is explicitly dropped.**
  - This makes it different from a local WITH clause or one-statement subquery.

Why Views Exist

Problems

- **Not all users should see all columns**.
  - Example: hide salary from a clerk.
- **Some users need a simpler logical interface.**
  - Example: list Physics sections in Fall 2017.
- **Repeated complex queries are error-prone**.
  - A stored query definition reduces repeated SQL text.

View design goals

- **Security / restriction**
  - Expose only the data that should be visible.
- **Abstraction**
  - Present a user-oriented logical relation.
- **Reuse**
  - Give a complex query a stable name.

Source: DSC 7e, Ch.4 §4.2, p.137

Generic CREATE VIEW Form

```
CREATE VIEW v AS
<query expression>;
```

- **v is the view name.**
  - It becomes a schema-level object.
- **The query expression defines the view contents.**
  - It can be any legal query expression supported by the DBMS.
- **The view can be referenced wherever a relation name may appear.**
  - The query processor then uses the view definition during execution.

Example

```
CREATE VIEW faculty AS
SELECT ID, name, dept_name
FROM instructor;
```

- The base relation is instructor.
  - It contains ID, name, dept_name, and salary.
- The view exposes **only three attributes**.
  - Salary is not in the view result.
- A clerk can be given access to faculty instead of instructor.
  - This is a logical interface and an authorization boundary.



View Definition

CORE

Element	Meaning	Exam question
View name	The logical relation exposed to users	What relation name will later queries reference?
SELECT list	Attributes visible through the view	Which base attributes are hidden?
FROM clause	Base relations or other views used to compute it	What dependencies does the view have?
WHERE/GROUP BY/etc.	Conditions and transformations defining the result	Is the view likely updatable?

A view is a contract: it names what users can see, not necessarily how the data is physically stored.

Source: DSC 7e, Ch.4 §4.2.1–§4.2.2

Example: physics_fall_2017

```
CREATE VIEW physics_fall_2017 AS
SELECT course.course_id, sec_id,
building, room_number
FROM course, section
WHERE course.course_id =
section.course_id
  AND course.dept_name = 'Physics'
  AND section.semester = 'Fall'
  AND section.year = 2017;
```

- **This view hides a join and several selection conditions.**
  - Users see a simple relation name.
  - The underlying query can remain centralized.
- **The view is still virtual.**
  - If course or section changes, the next use of the view reflects the current base data.
- **This is a good example of abstraction, not just security.**
  - The view matches a user-oriented task.

Querying a View

```
SELECT course_id
FROM physics_fall_2017
WHERE building = 'Watson';
```

- **The view name appears in the FROM clause.**
  - SQL treats it like a relation name.
- **The DBMS evaluates the view definition together with the outer query.**
  - Conceptually: compute view result, then filter building = Watson.
  - Implementation may use query rewriting or merging.
- **The user does not need to rewrite the original join.**
  - This reduces repeated SQL and improves readability.

Source: DSC 7e, Ch.4 §4.2.2, p.139

Explicit Attribute Names

```
CREATE VIEW departments_total_salary(dept_name,total_salary)
AS
    SELECT dept_name, SUM(salary)
    FROM instructor
    GROUP BY dept_name;
```

- Why the column list is useful:
  - SUM(salary) does not automatically give a stable meaningful column name.
  - Explicit names make downstream queries clearer.
- This view is an aggregate view.
  - It summarizes base rows by department.
  - It is useful for read queries, but it is not a simple updatable view.

Regular View Evaluation



- The view relation contains the result of the defining query at a given time.
  - For a regular view, the result is not stored as a persistent table.
- When the view is used, the DBMS must derive tuples from the current base data.
  - The logical meaning is simple; the physical implementation may be optimized.
- This is why regular views do not automatically solve performance problems.
  - They primarily support abstraction and authorization; performance depends on the final execution plan.

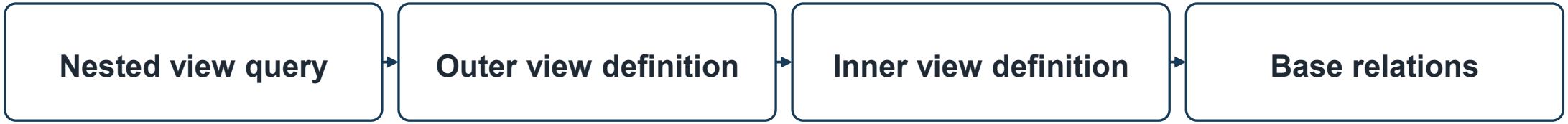
Nested Views

```
CREATE VIEW physics_fall_2017_watson
AS
    SELECT course_id, room_number
    FROM physics_fall_2017
    WHERE building = 'Watson';
```

- **A view definition may refer to another view.**
  - This is useful for building layered logical interfaces.
- **The DBMS can conceptually expand the nested view.**
  - `physics_fall_2017_watson` ultimately depends on course and section.
- **Design caution:**
  - Deep view chains can hide dependencies and make debugging harder.

View Expansion

```
-- Query over a nested view
SELECT course_id, room_number
FROM physics_fall_2017_watson;
```



- **The expanded query may contain the original joins and conditions.**
  - A view can hide query complexity from the user, but not from the DBMS.
- **The optimizer may transform the final expression.**

View vs Base Table vs Temporary Table

CORE

Object	Stores rows?	Lifetime	Main use
Base table	Yes	Persistent until dropped	Store application data
Regular view	No persistent result	Persistent definition until dropped	Logical interface over base data
Temporary table	Yes	DBMS-specific; in MySQL, session-specific	Intermediate or session-local data
Materialized view	Yes, if supported	Persistent stored result	Precompute expensive query results

Exercise 4.6 - View for GPA

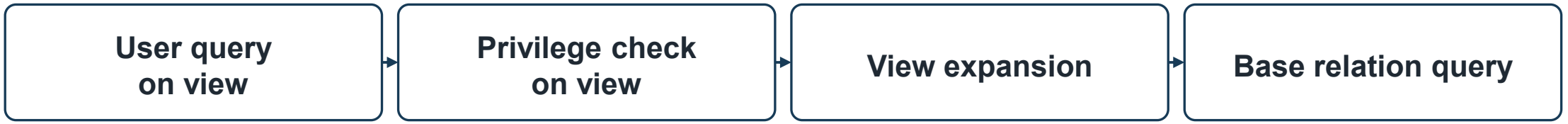
Define a view `student_grades(ID, GPA)` giving the grade-point average of each student. It must handle NULL grades correctly.

- **Given relations:**
  - `takes`(ID, course_id, sec_id, semester, year, grade)
  - `course`(course_id, title, dept_name, credits)
  - `grade_points`(grade, points)
- **Requirement:**
  - Use numeric grade points.
  - Do not count credits from course offerings whose grade is NULL.
- **In-class:**
  - Write the FROM/JOIN part first.
  - Then write the numerator and denominator of GPA.

Part II - Views and Authorization

Views can define what a user is allowed to see or modify.

View as an Authorization Boundary



- **A user may be granted access to a view instead of direct access to the base relation.**
  - Example: SELECT on faculty but not on instructor.salary.
- **The system must check authorization before replacing the view by its definition.**
  - Otherwise the expanded query might appear to require access to hidden base objects.
- **This is why views are not just query abbreviations.**
  - They can be part of database security design.

Example: geo_instructor

```
CREATE VIEW geo_instructor AS
SELECT *
FROM instructor
WHERE dept_name = 'Geology';
```

- A staff member should see salaries only for Geology instructors.
  - Direct access to instructor would reveal other departments.
- The view restricts rows by department.
  - The view result is a relation containing only Geology instructor tuples.
- **Authorization can be granted on the view.**
  - The user queries geo_instructor, not instructor directly.

Privilege Reasoning: Creator vs User

Role	What must be true?
View creator	Must have sufficient privileges on objects used in the view definition.
View user	Must have the appropriate privilege on the view object itself.
Privilege grantor	Can grant only privileges that do not exceed what the system allows the creator/owner to grant.
DBMS	Must enforce authorization according to the view definition and security context.

A view does not magically bypass security. It defines a controlled interface that the DBMS authorizes.

Source: DSC 7e, Ch.4 §4.7.3, p.169

MySQL Security Context for Views

```
CREATE
   [DEFINER = user]
   [SQL SECURITY { DEFINER | INVOKER }]
  VIEW view_name AS select_statement;
```

- **DEFINER** identifies the MySQL account associated with the view definition.
  - If omitted, it defaults to the account executing CREATE VIEW.
- SQL SECURITY controls privilege checking at invocation time.
  - **DEFINER**: check privileges against the definer account.
  - **INVOKER**: check privileges against the invoking account.
- Do not treat this as standard SQL syntax.
  - MySQL documents these clauses as MySQL extensions.

Source: MySQL 8.4 Reference Manual, §15.1.23 CREATE VIEW; §27.6 Stored Object Access Control

Part III - Views in MySQL

Syntax, lifecycle, metadata, and dependency behavior in MySQL 8.x.

- MySQL supports views as stored queries that act as virtual tables.
- MySQL adds implementation-specific clauses such as ALGORITHM and SQL SECURITY.
- View definitions and dependencies must be managed by the database administrator.

Source: MySQL 8.4 Reference Manual, §15.1.23 and §27.5

MySQL CREATE VIEW Syntax — Core Form

```
CREATE [OR REPLACE] VIEW view_name
[(column_list)]
AS select_statement
[WITH [CASCADED | LOCAL] CHECK OPTION];
```

- CREATE VIEW creates a new view object.
  - OR REPLACE replaces an existing view definition.
- column_list is optional but recommended when expressions are used.
  - It gives stable names to view columns.
- CHECK OPTION constrains inserts/updates through an updatable view.
  - Discussed after view-update semantics.

MySQL CREATE VIEW Syntax - Extensions

```
CREATE
  [ALGORITHM = {UNDEFINED | MERGE |
TEMPTABLE}]
  [DEFINER = user]
  [SQL SECURITY { DEFINER | INVOKER }]
  VIEW view_name [(column_list)]
  AS select_statement;
```

- ALGORITHM controls how MySQL processes the view.
  - MERGE attempts to merge the view definition into the outer query.
  - TEMPTABLE materializes the view result into a temporary table during execution.
- DEFINER and SQL SECURITY control privilege checking.

View Lifecycle Commands in MySQL

```
CREATE VIEW v AS SELECT ...;

ALTER VIEW v AS SELECT ...;

CREATE OR REPLACE VIEW v AS
SELECT ...;

SHOW CREATE VIEW v;

DROP VIEW v;
```

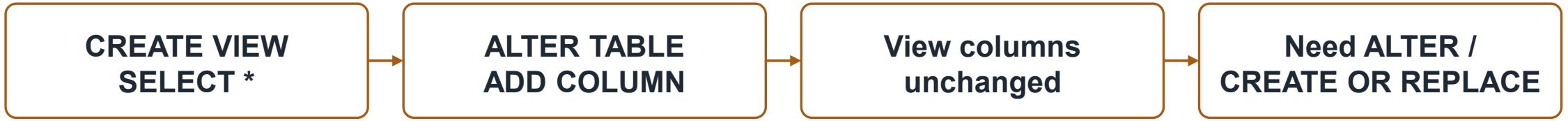
- Lifecycle management is part of DBMS administration.
  - A view is not just a one-off query in an application.
- SHOW CREATE VIEW is important for lab evidence.
  - It confirms the stored definition, security type, and options.
- ALTER VIEW changes a view definition that already exists.
  - The user must have appropriate privileges.

View Metadata in MySQL

```
SELECT table_schema, table_name,
       check_option, is_updatable,
       definer, security_type
FROM information_schema.views
WHERE table_schema = 'sakila';
```

Column	What it tells you
CHECK_OPTION	Whether CHECK OPTION is defined.
IS_UPDATABLE	Whether MySQL reports the view as updatable.
DEFINER	The account associated with view definition.
SECURITY_TYPE	DEFINER or INVOKER.

View Definitions Are Frozen in MySQL



A view is an interface. Interface stability can be useful, but hidden dependencies must be managed.

- MySQL freezes the view definition at CREATE VIEW time.
  - A view defined as SELECT * does not automatically include columns added later to the base table.
- If an underlying column or table is dropped, the view may become invalid.
  - The error appears when the view is used.
- DBA implication:
  - Avoid SELECT * in durable view interfaces.
  - Track dependencies before altering base tables.

Checking View Dependencies

```
CHECK TABLE view_name;

-- CHECK TABLE can report view
problems such as
-- references to dropped or renamed
underlying tables.
```

- A view definition may reference base tables, other views, and routines.
  - Changes to those objects can make the view unusable.
- CHECK TABLE can be used to check views for definition problems.
  - This is a lightweight administrative check after schema changes.
- Project implication:
  - If your application depends on views, include them in migration testing.

Safe Dependency Demo

```
CREATE TABLE view_demo_base (id INT PRIMARY KEY, a INT);
CREATE VIEW view_demo AS SELECT * FROM view_demo_base;

ALTER TABLE view_demo_base ADD COLUMN b INT;
SHOW CREATE VIEW view_demo;

ALTER TABLE view_demo_base DROP COLUMN a;
CHECK TABLE view_demo;

DROP VIEW view_demo;
DROP TABLE view_demo_base;
```

- Purpose:
  - Observe frozen definition and invalid dependency behavior safely.
- Why use a demo table?
  - Do not alter Sakila base tables for this test.
- Report evidence:
  - SHOW CREATE VIEW output.
  - CHECK TABLE output after dropping a referenced column.

Sakila

Object	Used for
customer_list	Inspect an existing view and test update behavior.
actor_list	Create a join/aggregation-style reporting view.
sales_by_store	Reference pattern for a reporting view.
sales_by_customer	Create a derived reporting interface.

- Sakila is used because it has realistic relationships.
  - customer, address, city, country, actor, film, payment, rental.
- The lab focuses on view behavior, not just syntax.
  - You must inspect definitions, test updates, and explain results.
- Do not modify data permanently unless instructed.
  - Use START TRANSACTION and ROLLBACK for experiments.

Example: staff_list

```
CREATE VIEW staff_list AS
SELECT s.staff_id, s.first_name, s.last_name,
       a.address, c.city, co.country
FROM staff AS s
JOIN address AS a ON s.address_id =
a.address_id
JOIN city AS c ON a.city_id = c.city_id
JOIN country AS co ON c.country_id =
co.country_id;
```

- This view hides a multi-table join.
  - Applications can query staff_list directly.
- It should be treated as a read interface.
  - Join views often have limited update behavior.
- Avoid SELECT * in durable view definitions.
  - Specify columns explicitly to keep the interface clear.

Querying a Sakila View

```
SELECT first_name, last_name, address
FROM staff_list
ORDER BY last_name, first_name;

SHOW CREATE VIEW staff_list;
```

- What to observe:
  - The query is simpler than the underlying join.
  - SHOW CREATE VIEW reveals the stored definition.
- What not to assume:
  - The view does not store staff rows separately.
  - ORDER BY should be specified in the outer query when result order matters.

MySQL View Processing Algorithms

Algorithm	Meaning	Updatability implication
MERGE	Merge view definition into outer statement.	Can preserve updatability if other rules allow it.
TEMPTABLE	Retrieve view result into a temporary table during statement execution.	A temporary-table view is not updatable.
UNDEFINED	Let MySQL choose; it prefers MERGE if possible.	Depends on chosen processing.

Source: MySQL 8.4 Reference Manual, §27.5.2 View Processing Algorithms

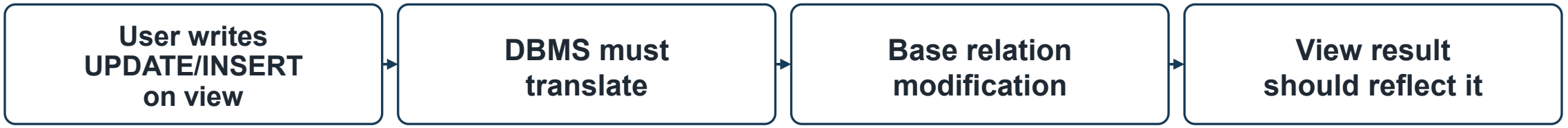
Part IV - Updating Views

Why SELECT through a view is easy, but INSERT/UPDATE/DELETE through a view is difficult.

- The DBMS must translate a view update into base-relation updates.
- That translation may be ambiguous or impossible.
- Updatable views are permitted only in restricted cases.

The View-Update Problem

CORE



- Querying a view is straightforward.
  - The DBMS computes a result from the view definition.
- Updating a view is harder.
  - A modification through a view must be translated into modifications on actual base relations.
- The translation must be meaningful.
  - If multiple base-table updates could explain the same view update, the operation is ambiguous.

Case 1: Inserting Through faculty

```
INSERT INTO faculty
VALUES ('30765', 'Green', 'Music');
```

- The view faculty omits salary.
  - But the base relation instructor requires a salary value in its schema.
- Two possible DBMS responses:
  - Reject the insertion.
  - Insert a tuple into instructor with salary set to NULL, if allowed.
- Exam point:
  - A view may hide columns, but base-table constraints still matter.

Case 2: Join-View Update Ambiguity

```
CREATE VIEW instructor_info AS
SELECT ID, name, building
FROM instructor, department
WHERE instructor.dept_name =
department.dept_name;

INSERT INTO instructor_info
VALUES ('69987', 'White', 'Taylor');
```

- The view row combines two base relations.
  - instructor contributes ID/name/dept_name.
  - department contributes building.
- A single view insert appears to require changes to two base tables.
  - But the missing join attribute dept_name cannot be inferred cleanly.
- The attempted base updates do not make the requested view tuple visible.
  - This is why join-view updates can fail.

Failed Join-View Insertion

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
69987	White	*null*	*null*

instructor

dept_name	*building*	*budget*
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Electrical Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000
null	Taylor	*null*

department

Relations *instructor* and *department* after insertion of tuples.

- What the figure shows:
  - The DBMS might insert one tuple with NULL dept_name into instructor.
  - It might insert one tuple with NULL dept_name into department.
- Why the desired view row is still absent:
  - The join condition is instructor.dept_name = department.dept_name.
  - NULL does not create a normal matching department name.
- Takeaway:
  - A view update is valid only if the base-relation update is well-defined.

Conditions for Updatable Views

An SQL view is updatable only under limited conditions.

- The FROM clause has only one database relation.
  - No multi-relation join in the view definition.
- The SELECT clause contains only attribute names of the relation.
  - No expressions, aggregates, DISTINCT, or computed columns.
- Attributes omitted from SELECT can be set to NULL.
  - They must not have NOT NULL constraints.
  - They must not be part of a primary key.
- The query has no GROUP BY or HAVING clause.
  - Grouped rows do not map one-to-one to base rows.

Source: DSC 7e, Ch.4 §4.2.4, p.142

Example: history_instructors

```
CREATE VIEW history_instructors AS
SELECT *
FROM instructor
WHERE dept_name = 'History';
```

- This view satisfies the basic book conditions.
  - It references one base relation.
  - It selects attributes without aggregation or DISTINCT.
  - It has no GROUP BY or HAVING.
- **But one problem remains**.
  - An inserted tuple may satisfy the base relation schema but not satisfy the view predicate.

Visibility After Insert

```
-- Insert through a History-only view
INSERT INTO history_instructors
VALUES ('25566', 'Brown', 'Biology',
100000);

-- Then query the same view
SELECT *
FROM history_instructors;
```

- The inserted tuple has dept_name = Biology.
  - It does not satisfy the view condition dept_name = History.
- Without CHECK OPTION, SQL may allow the base-table insertion.
  - The row is in instructor, but it is not visible through history_instructors.
- This is surprising for users.
  - They inserted through a view but cannot later see the inserted row through that view.

WITH CHECK OPTION

```
CREATE VIEW history_instructors AS
SELECT *
FROM instructor
WHERE dept_name = 'History'
WITH CHECK OPTION;
```

- CHECK OPTION enforces the view predicate on updates through the view.
  - An inserted row must satisfy the view WHERE condition.
  - An updated row must still satisfy the view WHERE condition.
- The Biology tuple is rejected.
  - It would not remain visible through the History view.
- Exam point:
  - CHECK OPTION connects update semantics with logical visibility.

MySQL Principle: One-to-One Mapping

For a MySQL view to be updatable, there must be a one-to-one relationship between rows in the view and rows in the underlying table.



- Updatable:
  - Each view row can be traced to exactly one base row.
- Not updatable:
  - A view row represents an aggregate, a DISTINCT row, or an ambiguous join result.

Source: MySQL 8.4 Reference Manual, §27.5.3 Updatable and Insertable Views

MySQL Non-Updatable Constructs

Construct in view definition	Why it blocks simple updating
Aggregate or window functions	A result row summarizes multiple base rows.
DISTINCT	Duplicates removed; mapping back to base rows is ambiguous.
GROUP BY / HAVING	Groups do not have a one-to-one base-row mapping.
UNION / UNION ALL	A row may originate from different branches.

MySQL Non-Updatable Constructs

Construct	Updatability issue
Certain joins	The DBMS may not know which base table/row to modify.
Subquery in SELECT list	Can break direct row mapping.
Reference to a nonupdatable view	Nonupdatability propagates.
Literal-only view	There is no underlying table to update.
ALGORITHM = TEMPTABLE	The temporary result cannot be updated back to base rows.

MySQL WITH CHECK OPTION

Option	Meaning
LOCAL	Check only the view being defined.
CASCADED	Check this view and underlying views.
Default	CASCADED if neither keyword is specified.

```
CREATE VIEW v AS
SELECT ...
FROM ...
WHERE condition
WITH CASCADED CHECK OPTION;
```

- Use CHECK OPTION when the view predicate defines the allowed interface.
  - Example: a department-specific view should not accept rows from another department.
- CHECK OPTION applies only to updatable views.
  - If the view is not updatable, the update fails before this check matters.

Source: MySQL 8.4 Reference Manual, §27.5.4 The View WITH CHECK OPTION Clause

Exercise: Can This View Be Updated?

```
CREATE VIEW dept_salary AS
SELECT dept_name, SUM(salary) AS
total_salary
FROM instructor
GROUP BY dept_name;
```

- Answer these questions:
  - Can we UPDATE total_salary for one department through this view?
  - Can we INSERT a new row into this view?
  - What base-table changes would be needed?
- Use the one-to-one mapping principle.
  - One aggregate row summarizes many instructor rows.

Part V - Materialized Views

A different design point: store query results to improve repeated expensive queries.

- Regular views are virtual.
- Materialized views store the query result if the DBMS supports them.
- The benefit must be weighed against storage and update-maintenance cost.

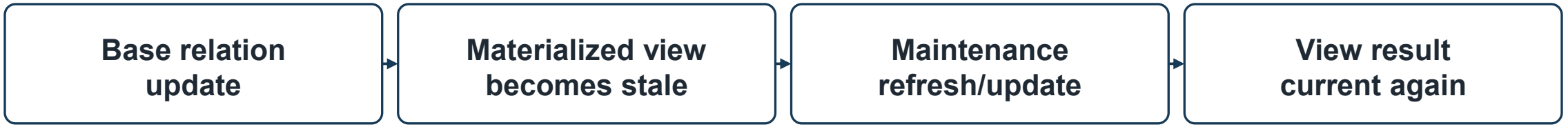
Regular View vs Materialized View

Aspect	Regular view	Materialized view
Stored data	No stored result; stores definition	Stores the computed result
Freshness	Reflects current base data at evaluation time	May need refresh/maintenance
Storage cost	Low	Additional storage
Query speed	Depends on final plan	Can speed repeated expensive queries
Update cost	No stored result to maintain	Maintenance overhead after base updates

Materialized views are a performance/maintenance trade-off, not merely a syntax variant.

Source: DSC 7e, Ch.4 §4.2.3, pp.140–141

Materialized View Maintenance



- Immediate maintenance:
  - Update materialized view as soon as base data changes.
- Lazy maintenance:
  - Refresh when the view is accessed.
- Periodic maintenance:
  - Refresh at scheduled intervals; result may be stale between refreshes.
- DBA decision:
  - Balance query performance, update overhead, storage, and freshness requirements.

When Materialized Views Are Useful

Good candidates

- Expensive repeated queries.
  - Example: large joins or aggregates run frequently.
- Aggregated result is much smaller than base relations.
  - Example: department-level totals over large transaction tables.
- Staleness is acceptable or refresh can be controlled.
  - Analytical dashboards may tolerate periodic refresh.

Poor candidates

- Frequently updated base tables.
  - Maintenance overhead may dominate.
- Strictly fresh results are required.
  - Stale materialized results may be unacceptable.
- Small or cheap queries.
  - No meaningful performance benefit.

MySQL Note: No Native Materialized View Statement

MySQL 8.4 does not provide a native CREATE MATERIALIZED VIEW statement.

- Do not write CREATE MATERIALIZED VIEW in MySQL lab reports.
  - That is PostgreSQL/Oracle-style syntax, not MySQL.
- Possible MySQL approximation:
  - Create a summary table with CREATE TABLE AS SELECT or INSERT ... SELECT.
  - Refresh it using scheduled jobs, application logic, or stored routines/events.
- Conceptual distinction:
  - A manually refreshed summary table is not the same as DBMS-native materialized-view maintenance.

Part VI - Lab L2A: Views in MySQL

Practice with Sakila and submit evidence, not just final SQL.

- Inspect existing views and metadata.
- Create new views from joins and aggregates.
- Test update behavior safely with transaction rollback.
- Explain results using textbook concepts and MySQL documentation.

Lab Setup

Requirement	Check
MySQL 8.x server	SELECT VERSION();
Client tool	MySQL Workbench, DBeaver, or mysql CLI
Sakila database loaded	SHOW DATABASES; USE sakila; SHOW TABLES;
Lab script file	Save all commands in L2A_views_<student_id>.sql
Screenshots or output file	Keep evidence for each task

Lab Task 0: Verify Sakila and Existing Views

```
USE sakila;

SHOW FULL TABLES
WHERE Table_type = 'VIEW';

SELECT table_name, is_updatable, check_option, security_type
FROM information_schema.views
WHERE table_schema = 'sakila'
ORDER BY table_name;
```

Lab Task A: Inspect customer_list

```
SHOW CREATE VIEW customer_list;

SELECT *
FROM customer_list
LIMIT 10;
```

- Check the view definition.
  - Which base tables are referenced?
  - Which columns are exposed to users?
- Check whether it is updatable.
  - Use INFORMATION_SCHEMA.VIEWS.
  - Then test carefully in Task B.
- Explaination:
  - Why does this view exist instead of asking users to write the join?

Lab Task B: Test Updates Through customer_list

```
START TRANSACTION;

-- Replace values and primary key with a safe test row
UPDATE customer_list
SET phone = '000-TEST'
WHERE ID = 1;

SELECT * FROM customer_list WHERE ID = 1;

ROLLBACK;
```

- Run only inside a transaction.
  - This prevents permanent changes to Sakila.
- Try updating address, phone, and city separately.
  - Record which operations succeed or fail.
- Explain using the view definition.
  - Does the update map to one base table or multiple base tables?
  - Does MySQL report the view as updatable?

Lab Task C: Create actor_list

```
CREATE OR REPLACE VIEW actor_list AS
SELECT a.actor_id,
       CONCAT(a.first_name, ' ', a.last_name) AS actor_name,
       GROUP_CONCAT(f.title ORDER BY f.title SEPARATOR ', ') AS films
FROM actor AS a
JOIN film_actor AS fa ON a.actor_id = fa.actor_id
JOIN film AS f ON fa.film_id = f.film_id
GROUP BY a.actor_id, a.first_name, a.last_name;
```

Lab Task D: Create sales_by_customer

```
CREATE OR REPLACE VIEW sales_by_customer AS
SELECT c.customer_id, c.first_name, c.last_name,
       SUM(p.amount) AS total_sales
FROM customer AS c
JOIN payment AS p ON c.customer_id =
p.customer_id
GROUP BY c.customer_id, c.first_name,
c.last_name;

SELECT *
FROM sales_by_customer
ORDER BY total_sales DESC
LIMIT 10;
```

- Important correction:
  - Do not rely on ORDER BY inside the view definition for persistent ordering.
  - Order the result in the outer SELECT.

Lab Task E: Metadata and Cleanup

```
SELECT table_name, is_updatable, check_option,
security_type
FROM information_schema.views
WHERE table_schema = 'sakila'
  AND table_name IN ('actor_list',
'sales_by_customer');

SHOW CREATE VIEW actor_list;
SHOW CREATE VIEW sales_by_customer;

DROP VIEW IF EXISTS actor_list;
DROP VIEW IF EXISTS sales_by_customer;
```

- Use metadata as evidence.
  - Do not only say “it works.”
  - Show how the DBMS records the view.
- Cleanup matters.
  - Dropping lab-created views avoids polluting the shared database.

Lab Task F: Dependency Sandbox

LAB

- Do this on a small demo table, not on Sakila base tables.
  - Create view_demo_base and view_demo.
  - Alter the base table to add and then drop a referenced column.
- Observe:
  - SHOW CREATE VIEW after adding a new column.
  - CHECK TABLE after dropping a referenced column.
- Explain:
  - What does “view definition is frozen” mean in this example?
  - Why is dependency checking part of DBMS administration?

Source: MySQL 8.4 Reference Manual §15.1.23; §15.7.3.2 CHECK TABLE

Part VII - Beyond the Core

Optional directions for stronger projects and independent exploration.

- Advanced topics are not core exam unless announced.
- They show that the same view concept is implemented differently across DBMSs.
- Use official documentation for project extensions.

Beyond MySQL: PostgreSQL Materialized Views

- PostgreSQL supports CREATE MATERIALIZED VIEW.
  - The query is executed to populate stored result data.
  - The stored query definition is remembered for later refresh.
- Core project question:
  - Compare a regular view and a materialized view for a repeated aggregate query.
  - What is stored, when is it refreshed, and what freshness trade-off appears?
- Official references:
  - https://www.postgresql.org/docs/18/rules-materializedviews.html
  - https://www.postgresql.org/docs/18/sql-creatematerializedview.html

Beyond MySQL: SQL Server Indexed Views

- SQL Server provides indexed views.
  - An indexed view stores computed data like a materialized view.
  - A unique clustered index is created on the view.
- Best fit:
  - Queries that aggregate many rows and are run repeatedly.
- Poor fit:
  - Base data that changes frequently, because maintenance cost increases.
- Project exploration:
  - Use one reporting query and compare regular view vs indexed/materialized design.
- Official reference:
  - https://learn.microsoft.com/en-us/sql/relational-databases/views/views

Beyond MySQL: Oracle Updatable Views

- Oracle documents inherently updatable views and INSTEAD OF triggers.
  - A view can be inherently updatable when columns map to one base table and restricted constructs are absent.
  - An INSTEAD OF trigger can define DML behavior for a view.
- Project exploration:
  - When a join view is not inherently updatable, can a trigger define a safe update rule?
- Official reference:
  - https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/CREATE-VIEW.html

Further Reading & Resources

Purpose	Resource
Required reading	DSC 7e Ch.4 §4.2 Views, pp.137–143; §4.7.3 Authorization on Views, pp.169–170.
MySQL implementation	MySQL 8.4 Manual §15.1.23 CREATE VIEW; §27.5 Using Views; §15.7.3.2 CHECK TABLE.
Advanced materialized views	DSC 7e Ch.16 §16.5 Materialized Views, pp.778–783.
PostgreSQL extension	PostgreSQL 18 docs: Materialized Views and CREATE MATERIALIZED VIEW.
SQL Server extension	Microsoft Learn: Views and Indexed Views.
Oracle extension	Oracle Database SQL Language Reference: CREATE VIEW and updatable views.